How Large Language Models Understand Your Writing

Large Language Models (LLMs) are types of artificial intelligence (AI) programs trained on extensive datasets – hence the name "large" – which enables them to recognize, understand, and create natural language text, among other tasks. Thanks to their significant contributions to advancing generative AI, LLMs have recently gained widespread recognition. They have also become a focal point for organizations looking to integrate artificial intelligence into a variety of business operations and applications.

LLMs learn to comprehend and process human language and other complex data through exposure to massive amounts of examples, often meaning thousands or millions of gigabytes worth of text from all over the internet. These models leverage deep learning to figure out the relationships between characters, words, and sentences by probabilistically analyzing unstructured data. It allows them to identify different types of content autonomously, without needing direct human guidance.

Whether it is to understand questions, craft responses, classify content, complete sentences, or even translate text into a different language, these AI models can be tailored to solve specific problems in different industries. Just like super readers inside a giant library filled with books, they absorb tons of information to learn how language works. In this article, we will dive in to explore the fascinating world of Large Language Models and how they work inside.

Major Features of Large Language Models

1 Large

- Huge amount of data for training the models (petabyte range)
- Hyperparameter count – The knowledge and skills that the machine learns and remembers after being trained

2 General Purpose

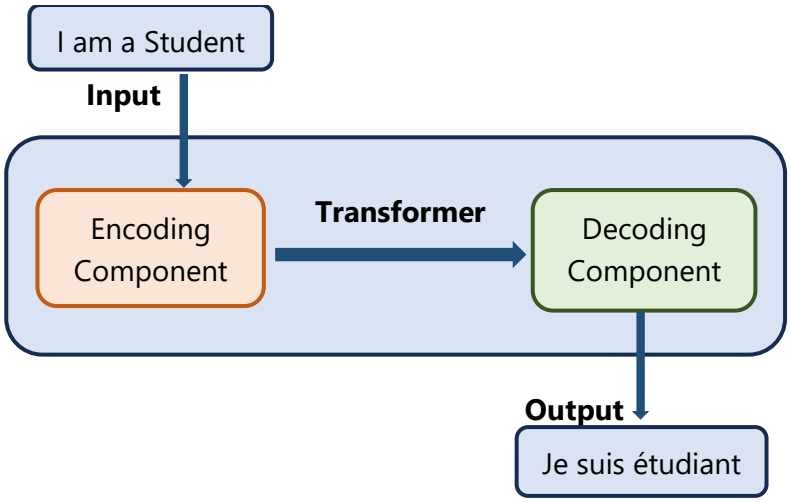
- The ability of LLMs to solve common problems, given the universal nature of human language and the resource limitations (Only a handful of companies are able to develop such complex models, aka foundational models that others can use)

3 Pre-trained, fine-tuned, and Multimodal

- Pre-trained on huge datasets for a general purpose
- Fine-tuned for specific tasks, using a relatively small size of field datasets – such as retail, finance, medical, or entertainment data
- Can be multimodal which combines text and visual information
- The performance grows continuously after adding more information and parameters

Core Mechanics of Large Language Models

At the heart of LLMs, we encounter the transformer model, which is crucial for understanding how these models operate. Transformers are built with an **encoder** and a **decoder**, processing data by breaking down inputs into tokens. They perform complex mathematical calculations to analyze the relationships between these tokens, and therefore come up with an outcome. In essence, the encoder “encodes” the input sequence and passes it to the decoder, which learns how to “decode” the representations for a relevant task.



Transformers enable a computer to recognize patterns similar to human cognition. These models leverage self-attention mechanisms, allowing them to learn more rapidly compared to older models, such as long short-term memory (LSTM) models. The self-attention mechanism allows them to process each segment of a word sequence while considering the context provided by other words within the same sentence.

To illustrate the concept of self-attention with a practical example, let’s look at how a transformer model tackles the task of translating a sentence. Imagine we are translating the sentence: "*The cat sat on the mat.*"

Input Encoding: The first step involves converting the input sentence into a series of word embeddings. Every word gets turned into a vector that represents its semantic significance in a high-dimensional space. Word embedding effectively captures a word's meaning, ensuring that words positioned closely within the vector space share similar meanings.

```
1. Example: [Embedding for 'The', Embedding for 'cat', Embedding for
   'sat', Embedding for 'on', Embedding for 'the', Embedding for 'mat',
   Embedding for '.']
```

Generating Queries, Keys, and Values: Following that, the self-attention mechanism produces three different forms of input embeddings: queries, keys, and values. These are created by linear transformations of the original embeddings and play a key role in computing the attention scores.

```
Queries: [Query for 'The', Query for 'cat', Query for 'sat', Query for 'on',
Query for 'the', Query for 'mat', Query for '.']

Keys: [Key for 'The', Key for 'cat', Key for 'sat', Key for 'on', Key for
'the', Key for 'mat', Key for '.']

Values: [Value for 'The', Value for 'cat', Value for 'sat', Value for 'on',
Value for 'the', Value for 'mat', Value for '.']

Random data:
 Queries:
[[0.23,0.4,.67,....],[0.4,0.6,.67,....],[0.2,0.2,.67,....],[0.5,0.3,.8,....],
[0.1,0.4,.67,....], [0.2,0.4,.67,....],[0.7,0.4,.6,....]]

 Keys:
[[0.1,0.4,.5,....],[0.2,0.4,.67,....],[0.3,0.4,.67,....],[0.4,0.4,.67,....],
[0.5,0.4,.67,....], [0.6,0.7,.8,....],[0.6,0.4,.8,....]]

 Values:
[[0.4,0.5,.67,....],[0.23,0.4,.5,....],[0.23,0.4,.8,....],[0.23,0.4,.45,....]
,[0.23,0.4,.9,....],[0.23,0.4,.6,....],[0.23,0.4,.10,....]]
```

Determining Attention Scores: The process involves computing the dot product between a query and all the keys, which generates the attention scores. These scores indicate the significance or relevance of each word in relation to the word currently under consideration.

```
Randoms Scores Example:
Attention scores for 'The': [0.9,0.7,0.5, 0.4,0.45,0.56,0.23]
Attention scores for 'cat': [0.6,0.5,0.7, 0.23,0.44,0.58,0.23]
.....
Attention scores for '.': [0.3,0.5,0.9, 0.4,0.45,0.56,0.23]
```

Using SoftMax: The attention scores are processed through the SoftMax function, transforming them into probabilities. This step guarantees that the attention weights total 1, reflecting the proportional significance of each word concerning the word being analyzed.

```
Example: Softmax of attention scores 'The': [0.29, 0.1, 0.12, 0.14, 0.1, 0.1,
0.14]
```

Calculating Weighted Sum: The attention probabilities obtained from the SoftMax function are then utilized to calculate a weighted sum of the values. This final vector provides a context-sensitive depiction of the current word, factoring in its connections with the rest of the words in the sequence.

```
Example: Context-aware representation : [0.29 * Value for 'The' + 0.1 * Value
for 'cat' + 0.12 * Value for 'sat' + …]
```

The resulting representation captures the contextual significance of all words, considering their associations with other words in the sentence – which enhances the model's predictive capabilities. After the multiplication with the values, you end up with a 2D matrix. Then, the language model selects the option with the greatest likelihood. This method is known as the "Greedy Approach," where there is a lack of creativity as the model consistently opts for the same word. On the contrary, the language model can also make its choice randomly, leading to more creative outcomes.

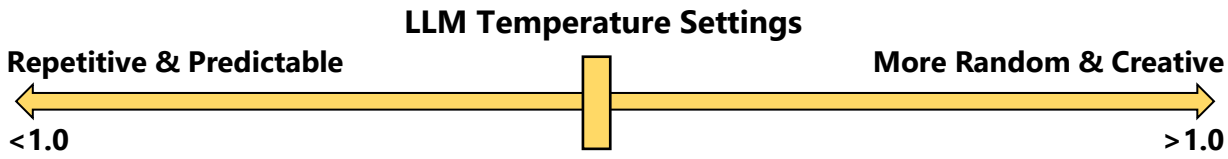
In the sentence: *The cat sat on*…, the next word is most likely going to be "*the*". However, if we choose randomly among other options, we can get something like "*bottle*" or "*plate*", which obviously has a much lower probability. To control the creativity levels, we need to adjust the **temperature parameter** that influences the models' results. The temperature is a numerical value (often set between 0 and 1, but sometimes higher) that is critical for **fine-tuning** the model's performance.

Adjusting Temperature: This parameter is directly incorporated into the SoftMax function. In a nutshell, If we are after the same old answers with zero creativity, we have to decrease the Temperature. But if we want something fresher and more out-of-the-box, then we have to increase the parameter value. Following, we describe how different temperature values modify the probability distribution of the next word in a sentence:

- **Low Temperature (Below 1.0)** – Dialing the temperature below 1 tunes the model towards more predictable and less diverse outputs. It narrows down the model's choices, often opting for the most likely word next, which might make the text seem less creative or varied, and maybe a bit more mechanical. This setting is great for when you want straightforward, less surprising answers.

- **High Temperature (Above 1.0)** – Cranking the temperature above 1 introduces more unpredictability to the text generation. The model ventures beyond the obvious choices, picking less likely words, which can make the content more diverse and potentially more creative. But beware, this can also lead to more mistakes or even nonsensical bits, as the model strays further from its training data's probability paths.

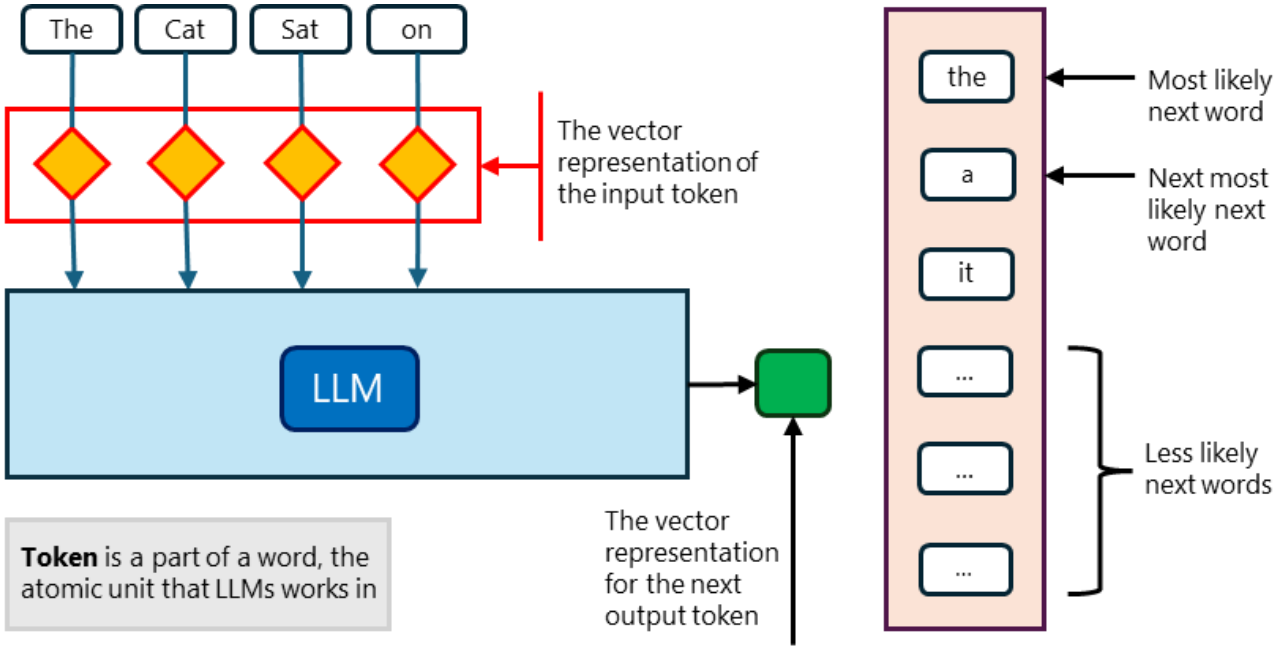
- **Setting the Temperature to 1.0** – Often the go-to middle ground, a temperature of 1.0 seeks to strike a balance between the predictable and the unpredictable. In this setting, the model produces text that is a mix, neither shifting too much into monotony nor into chaos, which reflects the probability distribution it was trained on.



Types of Large Language Models

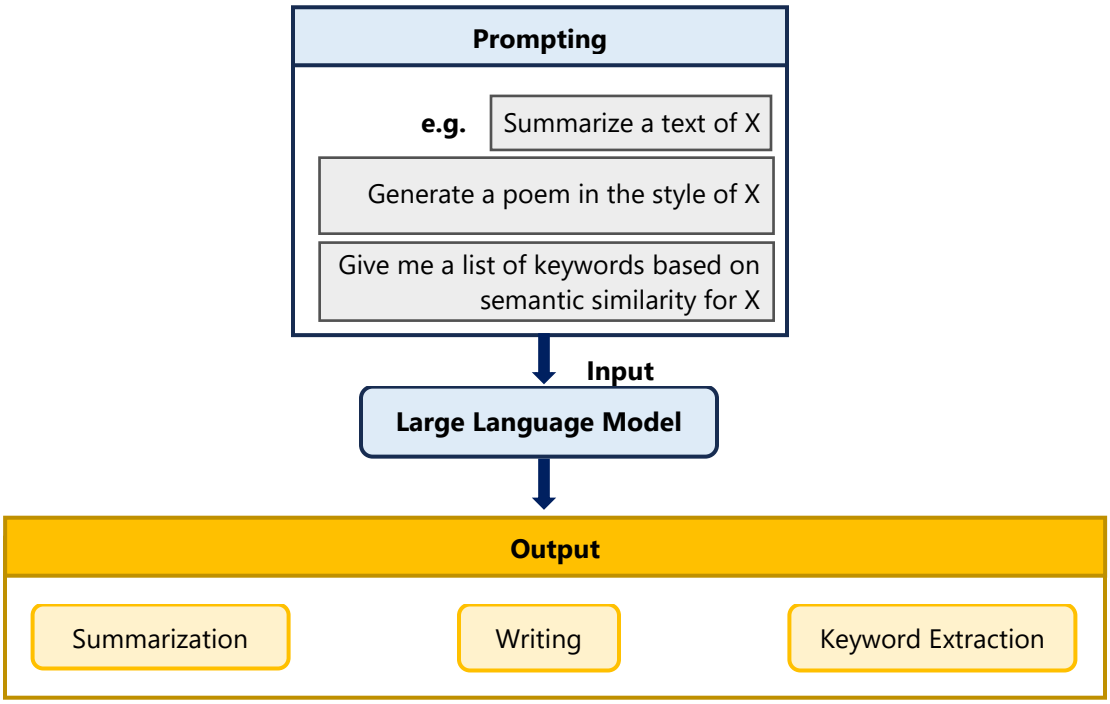
The terminology we use to categorize different kinds of large language models keeps changing, as they become extremely flexible and adaptable. Here are the three main types of LLMs that you will hear about often:

1. **Generic Language Models**: This model is trained to predict the next word (also called **token**) based on the language in the training data. In the following example, the phrase "The cat sat on…" the next word should be "the", which is the most likely next word. Think about this type of LLM as a really sophisticated autocomplete function in search.



These models calculate the likelihood of a token or a series of tokens appearing within a longer sequence of tokens. Considering a token as a word, a language model predicts the chances of various words or word sequences filling in the blank.

2. **Instruction Tuned**: This type of model predicts a response to the instructions given in the input. As an example, if you ask to summarize a text of "x", and generate a poem in the style of "x", give me a list of keywords based on semantic similarity for "x".



This approach serves a purpose similar to fine-tuning, in which it trains a model for a particular task via few-shot or zero-shot prompting. A **prompt** is essentially an instruction provided to an LLM. Few-shot prompting teaches the model of predicting outcomes by presenting examples. For instance, in a sentiment analysis task, a few-shot prompt might appear as follows:

```
1. Customer review: This plant is so beautiful!
2. Customer sentiment: positive

3. Customer review: This plant is so hideous!
4. Customer sentiment: negative
```

The language model, realizing the semantic implication of "hideous," and given the context of an opposite example, would discern that the customer sentiment in the second instance is "negative." On the other hand, zero-shot prompting does not rely on examples to guide the language model's response to inputs. Rather, it phrases the prompt as "The sentiment in 'This plant is so hideous' is...," directly signaling the task to be performed by the model without offering examples for problem-solving. Also, we can classify the text as neutral, negative, or positive in a given context.

Instructions →	Classify the text into neutral, negative, or positive	
	Text: I think the food was okay ←	**Input data**
Output indicator →	*Sentiment:*	

3. **Dialog Tuned**: This type of model is trained to have a dialog by the next response. Dialog-tuned models are a special case of instruction-tuned, where requests are typically framed as questions to a chatbot. Dialog tuning is expected to be in the context of a longer back-and-forth conversation and typically works better with natural question-like phrasings.

Prompt Examples
[User] Is the comment "Do you like the weather?" ok or toxic? **[Bot]** Ok. **[User]** Can you briefly say why? **[Bot]**
Model Output
It is just a question about the weather, people are not usually upset by that

This type of LLM can include a chain of thought reasoning, which is the observation that models are better at getting the correct answer when they first output text that explains the reason for the answer.

	Prompt Examples
Models are better at getting the right answer when they **first output** text that explains the reason for the answer →	**Q:** Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? **A:**

		Prompt Examples
The model is less likely to get the correct answer directly	➡	**Q:** Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? **A:** Let's think this through step by step.
Now the output is more likely to end with the **correct answer**	➡	**A:** Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Most Popular LLMs Today

Large language models have been decisive in the boom of **generative AI** technology witnessed in 2023. Most of these models are built on the transformer architecture, such as the Generative Pre-trained Transformer (GPT) series and Bidirectional Encoder Representations from Transformers (BERT). After its launch in 2022, ChatGPT (from OpenAI) rapidly gained a massive user base, attracting over 100 million users within just two months. This success sparked the release of numerous competing models from major corporations like Google, and Microsoft, and from the open-source community.

The landscape of LLMs includes a wide range of influential models, both historical and contemporary, which have either set the groundwork for the current frontrunners or are poised to significantly impact the future. Among these, some have shaped the direction of today's AI capabilities, while others, though possibly less recognized, hold the potential to drive forward the next wave of innovations. Below are some of the most significant large language models today, known for their natural language processing capabilities and their influence on the design of future models:

- **BERT:** Introduced by Google in 2018, BERT represents a series of LLMs built on transformer technology, capable of transforming data sequences into other data sequences. The architecture of BERT consists of a series of transformer encoders, totaling 342 million parameters. Initially pre-trained on a vast dataset, BERT was subsequently fine-tuned for particular tasks, including natural language inference and text similarity at the sentence level. In its 2019 update, Google Search leveraged BERT to enhance its understanding of search queries.

- **Gemini:** Gemini – Google's suite of LLMs – powers and shares its name with the company's chatbot, succeeding PaLM and coming in with a rebrand from Bard to Gemini. Unique for their multimodal capabilities, Gemini models can process not just text, but also images, audio, and video. It has been integrated into a large number of

Google's applications and products, offering the Ultra, Pro, and Nano variations. Ultra represents the largest and most sophisticated option, Pro serves as the intermediate model, and Nano is the smallest version, optimized for efficiency with on-device operations.

- **GPT-3.5:** GPT-3.5, an enhanced variant of GPT-3, comes with a reduced number of parameters and has been refined through reinforcement learning based on human feedback. This version powers the capabilities of ChatGPT. Among its variants, GPT-3.5 Turbo stands out as the most advanced, as per OpenAI's assessment. The training dataset for GPT-3.5 goes up until September 2021. Moreover, it was recently incorporated into the Bing search engine, although it has now been succeeded by GPT-4.

- **GPT-4:** Released in 2023, GPT-4 stands as the largest model within OpenAI's GPT lineup. Continuing the tradition, it is built on a transformer framework. However, its exact number of parameters remains undisclosed, with speculations suggesting it surpasses 170 trillion. OpenAI highlights GPT-4's multimodal capabilities, enabling it to understand and create content in both text and images, expanding beyond the mere text functions of its predecessors. Additionally, GPT-4 brings a novel feature to the table − a system message function, allowing users to define the tone of voice and specific tasks.\

- **Llama:** In 2023, Meta introduced its Large Language Model, Llama, marking its incursion into the LLM arena. Boasting up to 65 billion parameters in its largest iteration, Llama initially served an exclusive group of researchers and developers before transitioning to an open-source model. Built on the transformer model, Llama's training involved a diverse group of public datasets, such as CommonCrawl, GitHub, Wikipedia, and Project Gutenberg. Following an inadvertent leak, Llama gave rise to a series of offshoots, including Vicuna and Orca, expanding its legacy within the AI domain.

- **Orca:** Microsoft's Orca, with its 13 billion parameters, is compact enough to be operated on a laptop. It seeks to build upon the progress of other open-source models by replicating the reasoning capabilities exhibited by large language models. Despite having far fewer parameters, Orca matches GPT-4's performance and equals GPT-3.5 in numerous tasks. The foundation of Orca is the 13 billion parameter iteration of LLaMA.

- **xAI Grok:** In the middle of March 2024, X.ai unveiled Grok-1 AI chatbot, the most substantial "open-source" large language model (LLM) to date, featuring 314 billion parameters. This makes it the biggest open-source model available, significantly surpassing earlier models such as Falcon 180B, which had 180 billion parameters. Grok-1 is built on a Mixture of Experts (MoE) model, which activates only 25% of its weights for any specific token during inference. Official statements indicate that it has not undergone any fine-tuning for particular uses such as conversational agents.

Overcoming Challenges

Most of the information that goes into training cutting-edge Large Language Models comes from just gathering texts from all over the internet – such as the Common Crawl dataset that grabs data from over 3 billion web pages. Within those massive data hauls, there is a ton of private information from all types of people who have something about them online. Information that might be either accurate or inaccurate, and may even contain outright false information. This type of scenario brings up data protection and privacy challenges that are exceptionally difficult to address.

Furthermore, without adequate safeguards, the outputs generated by LLMs could disclose sensitive or private data found in the training datasets, potentially leading to actual or possible **data breaches**. The upside is that Large Language Models (LLMs) are not built with flaws that make them prone to leaking private data right off the bat. A big model is not just going to start throwing out private information because of how it is made. Instead, the risk of data breaches comes down to how the people running the model manage and use it.

On the other hand, Large Language Models can occasionally 'hallucinate,' generating false information that seems accurate. These hallucinations can result in the dissemination of incorrect, nonsensical, or misleading details about people, which could definitely harm an individual's reputation and influence decisions that concern them. Moreover, when LLMs are trained on biased datasets, they risk reinforcing or even exacerbating the biases in that data. This situation results in outputs that are discriminatory or unfair, possibly breaching the standard of equitable personal data processing.

The Path Forward

As Large Language Models (LLMs) evolve, they are expected to enhance across the board. Future versions will likely produce responses that are not only more coherent but also exhibit advanced capabilities in identifying and reducing bias, as well as in being more transparent. This progress promises to establish LLMs as dependable tools across various sectors such as finance, manufacturing, content generation, healthcare, and education.

The market is expected to grow in the number and diversity of LLMs available, providing organizations with a broader spectrum of options to pinpoint the most suitable LLM for their specific AI initiatives. Customizing these models is also set to become much simpler and more precise, enabling AI applications to be optimized for greater speed, efficiency, and productivity. In the future, the cost of large language models will decrease significantly, making it feasible for small businesses to tap into the benefits and capabilities of them.